

National Textile University, Faisalabad



Department of Computer Science

Name:	Maryam Shamshad
Class:	BSCS-A
Registration No:	23-NTU-CS-1047
Course Name:	Embedded Iot Systems
Submitted To:	Sir Nasir Mahmood
Date of Submission:	18 th December,2025

Assignment # 02

Question # 01

Part-A: Short Questions

1. What is the purpose of WebServer server(80); and what does port 80 represent?

WebServer server(80) creates a web server on the device ESP32 allows users to open a web page in a browser and interact with the device i.e. view data, control LEDs, etc. Port 80 is the default HTTP port. Whenever, we type the ESP32 IP address in a browser, the request automatically goes to port 80.

2. Explain the role of server.on("/", handleRoot); in this program.

It tells the web server what to do when someone opens the home page. "/" represents the root URL of the web server while handleRoot is a function that contains the code that run when specific page is requested. We can say that when a user opens the home page (/), the handleRoot function runs and shows the web page.

3. Why is server.handleClient(); placed inside the loop() function? What will happen if it is removed?

Server.handleClient() checks for new incoming requests from the browser. The loop function runs continuously. So, the server can constantly listen and respond to clients.

If it is removed, server will not process any requests, webpages will not load or refresh and the web server stops working.

4. In handleRoot(), explain the statement: server.send(200, "text/html", html);

Server.send(200, "text/html", html) is a statement that send requested page to the user.

Where, all there parameters are following

- **200** shows HTTP status code that means request was successful (OK).
- **"text/html"** tells the browser the type of content. Here, it means the response is an HTML web page.
- **html** a variable that contain the actual HTML content.

5. What is the difference between displaying last measured sensor values and taking a fresh DHT reading inside handleRoot()?

Last Measured Values:

- It shows already stored temperature and humidity values.
- The response is fast because the sensor is not read again.
- It avoids overloading the DHT sensor with frequent reads.

Fresh DHT Readings:

- It reads the latest temperature and humidity values.
- The response is slow because the sensor needs time to read.
- It can cause errors or unreliable readings if accessed too often.

Part-B: Long Question

1. ESP32 Wi-Fi connection process and IP address assignment

- The ESP32 connects to Wi-Fi using **WiFi.begin(ssid, password)** then **WiFi.status()** tells that connection is established or not.
- Once connected, the router automatically assigns an IP address to the ESP32.

- This IP address is shown on the serial monitor which is later used to access the web page in a browser.

2. Web server initialization and request handling

- A web server object is created using **WebServer server(80)**, where port 80 is the default HTTP port.
- The statement **server.on("/", handleRoot);** links the home URL (/) to the **handleRoot()** function.
- **server.begin()** starts the web server.
- Inside the **loop()** function, **server.handleClient()** continuously listens for incoming browser requests and responds to them.

3. Button-based sensor reading and OLED update mechanism

- A push button is connected to GPIO 5 and configured using **INPUT_PULLUP**.
- When the button is pressed, it goes from High to Low state:
 - ESP32 reads temperature and humidity from the DHT11 sensor using **readDHTValues()**.
 - It stores the readings in **lastTemp** and **lastHum**.
 - It updates the OLED display using **showOnOLED()** to show the latest values.

4. Dynamic HTML webpage generation

- The webpage is created dynamically inside the **handleRoot()** function using a String **html**.
- The page displays:
 - The temperature and humidity if valid data is available.
 - A message asking the user to press the button if no data has been recorded yet..
- The stored values (**lastTemp**, **lastHum**) are inserted into the HTML before sending it to the browser using **server.send(200, "text/html", html)**

5. Purpose of meta refresh in the webpage

- The line **<meta http-equiv='refresh' content='5'>** automatically refreshes the webpage every 5 seconds.
- This allows the browser to show updated sensor values without manual reloading.

6. Common issues in ESP32 webserver projects and their solutions

- **Wi-Fi not connecting:** Ensure correct SSID/password and stable signal.
- **Webpage not loading:** Confirm **server.handleClient()** is inside **loop()**.
- **Invalid DHT readings:** Avoid frequent reads, use button-based or timed readings.
- **Blank OLED:** Check I2C address (0x3C) and wiring.

Question # 02

Part-A: Short Questions

1. What is the role of Blynk Template ID in an ESP32 IoT project? Why must it match the cloud template?

- Blynk template id link your ESP32 code to template you created in the Blynk Cloud. It tells the cloud which template your device should use.
- If the template id in the ESP32 code doesn't match the one in the Blynk Cloud, the device cannot connect correctly, and the app may not show the right widgets or data.
- It also ensures proper synchronization between the physical device and the cloud dashboard.

2. Differentiate between Blynk Template ID and Blynk Auth Token.

Blynk Template ID:

- It tells the cloud which template and dashboard the device should use.
- It is same for all devices using the template.
- It ensures the device follows the correct layout and widgets.

Blynk Auth Token:

- It tells the cloud which device is connecting.
- It is unique for each device.
- It allows the device to send and receive data securely.

3. Why does using DHT22 code with a DHT11 sensor produce incorrect readings?

Mention one key difference between the two sensors.

Using DHT22 code with a DHT11 sensor produces incorrect readings because the two sensors have different data formats and timing. The code for DHT22 expects the DHT22's specific signal pattern, which DHT11 does not follow exactly, leading to errors.

DHT11 Sensor:

- Less accuracy, small temperature range and slow response.

DHT22 Sensor:

- High accuracy, large temperature range and fast response.

4. What are Virtual Pins in Blynk? Why are they preferred over physical GPIO pins for cloud communication?

Virtual Pins in Blynk (V0, V1...) are software-based pins used to send and receive data between the device and the Blynk Cloud, without relying on the physical GPIO pins of the hardware.

They preferred over physical pins because

- It is Flexible as no need of any external physical pins.
- Due to scalability, multiple virtual pins are used, even hardware GPIO pins are limited.
- It makes remote control and monitoring easy.

5. What is the purpose of using BlynkTimer instead of delay() in ESP32 IoT applications?

BlynkTimer:

- Blynk Timer is non-blocking that allows other code to run while waiting.
- It can run many timed tasks simultaneously.
- It keeps ESP32 responsive to cloud, sensors, and buttons.

delay():

- It is blocking that pauses the entire program.
- It only handles one delay at a time.
- It can freeze cloud communication and inputs.

Part-B: Long Question

1. Creation of Blynk Template and Datastreams

- A blynk template named “**dht**” is created in the blynk cloud.
- Following datastreams are configured inside the template.
 - Virtual pin **V0** = Temperature
 - Virtual pin **V1** = Humidity
- These datastreams tells how data from the esp32 will displayed on the blynk dashboard i.e.

2. Role of Template ID, Template Name, and Auth Token

- Blynk template id uniquely identifies the blynk template in the cloud. It ensures the device follows the correct layout and widgets as **#define BLYNK_TEMPLATE_ID "TMPL6UCKQ_P6D"**
- Blynk template name is the human readable name such as **#define BLYNK_TEMPLATE_NAME "dht"**
- Blynk auth token tells the cloud which device is connecting. It allows the device to send and receive data securely like **#define BLYNK_AUTH_TOKEN "S2P_gLWBb5E-DINBtsXmIOcndOq6xJnU"**

3. Sensor configuration issues (DHT11 vs DHT22):

- **#define DHTTYPE DHT22** tells that DHT22 is used in the code.
- If you are using DHT11 instead of DHT22 then false readings will show on the dashboard because defined sensor is DHT22 have high accuracy and wider range for temperature and humidity than the other.
- Therefore, you must change the sensor from DHT22 to DHT11 in the code.

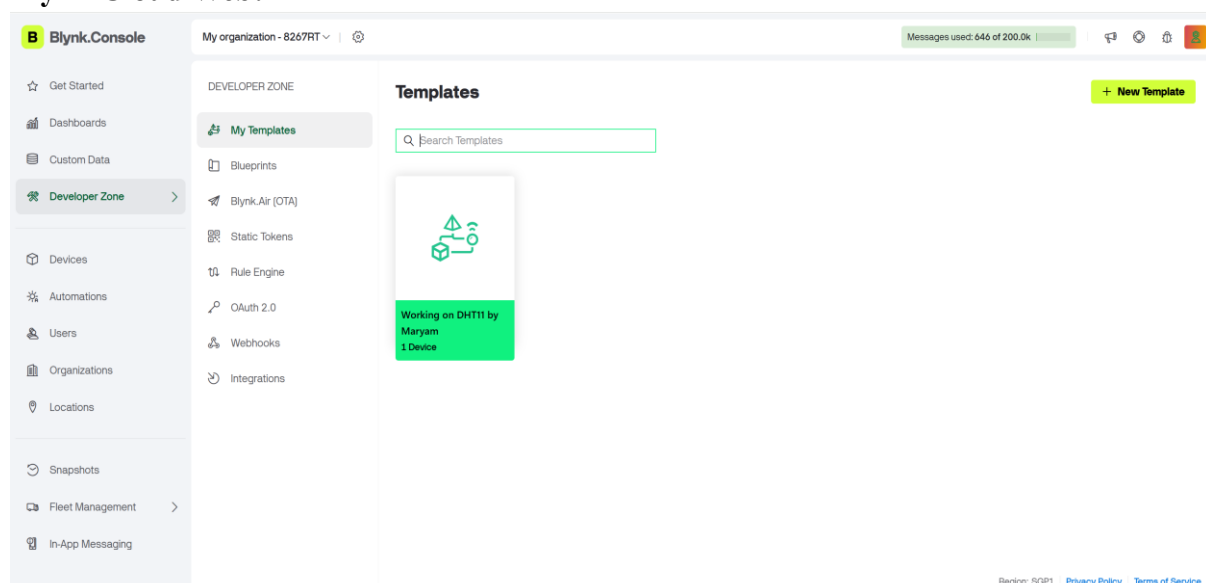
4. Sending data using Blynk.virtualWrite():

- The function **readAndDisplayAndSend()** perform following tasks:
 - It reads temperature and humidity from the sensor.
 - It displays values on the OLED.
 - It sends values to blynk cloud.
- Data can be send on blynk cloud by using **Blynk.virtualWrite()** that can be received on corresponding datastreams and shown on the dashboard as:
 - **Blynk.virtualWrite(V0, t);**
 - **Blynk.virtualWrite(V1, h);**
- Data can be send manually when the button is pressed and also by BlynkTimer.

5. Common problems faced during configuration and their solutions

- **ESP32 not connecting to blynk:** Ensure auth token, template id, Wi-Fi ssid and password are correct
- **No sensor data on blynk:** Check your virtual pin mapping (V0, V1) is correct
- **Incorrect readings:** Verify sensor type (DHT11).
- **Blank OLED:** Check I2C address (0x3C) and wiring.

Blynk Cloud Web:



Embedded Iot Systems

The screenshot shows the Blynk Console interface. The left sidebar contains navigation options: Get Started, Dashboards, Custom Data, Developer Zone (highlighted), Devices, Automations, Users, Organizations, Locations, Snapshots, Fleet Management, In-App Messaging, and Collapse. The main content area is titled 'Working on DHT11 by Maryam'. It features a 'Home' section with a table listing 1 device: DHT11, which is Online with an Auth Token 6Q-0 - The right sidebar shows 'Template settings' (ESP32, WIFI) and 'Firmware configuration' with a code block containing:

```
#define BLYNK_TEMPLATE_ID "TMPL66hM4sD_9"
#define BLYNK_TEMPLATE_NAME "Working on DHT11 by Maryam"
```

The screenshot shows the Blynk Console 'Devices' page. The left sidebar is the same as the previous screenshot, with 'Devices' highlighted. The main content area shows the 'DHT11' device status as Online. Below the status bar are two large gauge charts: 'Temperature' at 24°C and 'Humidity' at 40%. Both gauges have a scale from 0 to 100. The bottom right corner of the page indicates 'Region: SGP1' with links to 'Privacy Policy' and 'Terms of Service'.

Blynk Mobile App:

